

Moveo Coding Task - JaMoveo

We are building a collaborative web application where users can create or join rooms to jam together in real-time. The admin controls the session flow and displays songs with lyrics and chords for participants.

Task Description

Requirements

Home Page

- **Create Room:**
 - Users can create a new room and become the admin.
 - Generates a unique **Room Code** for others to join.
 - Redirects to the **Room Page** for the admin.
- **Join Room:**
 - Displays a list of active rooms.
 - Users can search for and select a room to join.
 - Redirects to the **Room Page** upon joining.

Room Page

1. **Player View:**
 - Displays a message: "**Waiting for the next song**" until the admin selects a song.
 - Updates to the **Live Page** when a song is selected.
 - Includes a list of connected users and their chosen instruments.
2. **Admin View:**
 - Displays a search bar with the title "**Search any song...**".
 - Allows the admin to search for songs by title or artist and proceed to the **Results Page**.

Results Page (Admin Only)

- Displays search results (song name, artist, and image if available).
- Admin selects a song, redirecting all users to the **Live Page**.

Live Page (Integrated into Room Page)

- **Singers:** Display only the song lyrics.
- **Other players:** Display both lyrics and chords.
- Includes:

- A floating toggle button to enable/disable automatic slow scrolling.
 - Large font size and high-contrast design for readability.
 - Admin-only controls:
 - **Quit:** Ends the session and redirects all users to the **Room Page**.
-

Technical Guidelines

1. **General:**
 - This task involves client-server and database components.
 - Socket implementation is mandatory for real-time updates.
 - Use the attached JSON song database as a hardcoded data source.
 2. **Database Entities:**
 - Rooms: Stores room-specific data (room code, admin, participants).
 - Users: No sign-up or login is required. Users specify their instrument when joining a room.
 3. **Frontend:**
 - Use React for the user interface.
 - Ensure a responsive and user-friendly design.
 4. **Backend:**
 - Use Node.js with Socket.IO for WebSocket communication.
 5. **Styling:**
 - Use Styled-Components, Tailwind CSS, or MUI for UI components.
 - Add cool and creative styling per your imagination.
 6. **Deployment:**
 - Deploy the app using Netlify, Vercel, or AWS.
 - Provide the deployed URL and clear instructions in the README for:
 - How to create a room.
 - How to join a room.
-

Deliverables

1. Deployed application with a working URL.
2. Clear README including:
 - Instructions to create and join rooms.
 - Explanation of admin controls in the **Room Page**.

Bonus Feature

- **Dynamic Data:** Replace hardcoded song data with dynamically fetched lyrics and chords from a website like **Tab4U**.

 C Em C
Hallelujah, hallelujah, hallelujah,
 G D G D
Hallelu-u-jah.

Live Page example for singer (only lyrics):

Hallelujah, Hallelujah
Hallelujah, Hallelujah

